Technical Architecture

This project implements a modern data + AI architecture combining Microsoft Fabric (Lakehouse + Warehouse) with a Retrieval-Augmented Generation (RAG) system and Text-to-SQL interface.

🔷 1. Architecture Overview

User (Natural Language Query)
↓
Python Application Layer
↓
LLM (GPT-4) + Vanna AI (Text-to-SQL)
↓
RAG Layer (Schema + Metadata Context)
↓
SQL Query Execution (Fabric SQL Endpoint)
↓
Microsoft Fabric (Warehouse + Lakehouse)
↓
OneLake Storage (Delta / Parquet)
↓
Data Sources (Clients, Assets, Transactions)

🧱 2. Core Components

- Data Ingestion Layer

Tool: Dataflow Gen2
Function:
Extract data from source systems (databases, files)
Load into OneLake (Lakehouse layer)
Supports scalable and low-code ingestion pipelines

- Data Storage Layer (OneLake)

Centralized storage using OneLake
Stores data in Delta/Parquet format
Organized into:
Lakehouse (Bronze/Silver layers) → Raw and cleaned data
Warehouse (Gold layer) → Structured analytical data

- Data Processing & Modeling Layer

Data transformation using:
Dataflows Gen2
SQL transformations
Structured modeling:
Fact tables → Transactions
Dimension tables → Clients, Assets, Portfolios
Creation of:
Aggregated tables (portfolio summaries)
Analytical views for reporting

- Data Serving Layer (SQL Endpoint)

Fabric SQL Endpoint / Data Warehouse
Enables:
T-SQL querying
Views and procedures stored
Acts as the primary interface for analytical queries

- Application Layer (Python)

Python scripts handle:
Query orchestration
API integration
Result processing
Uses SQL connection string to interact with Fabric

- AI Layer (LLM + RAG)

LLM (GPT-4 / Azure OpenAI):
Converts natural language → SQL queries
Generates explanations for results
RAG (Retrieval-Augmented Generation):
Provides schema-aware context to LLM

- Text-to-SQL Engine (Vanna AI)

Convert user queries into SQL
Enhances LLM with:
Schema understanding
Query accuracy
Reduces hallucination by grounding queries in actual database structure

- Visualization Layer

Vanna AI dashboard for:
Query outputs
Data insights
Can be extended to Power BI for enterprise reporting

3. End-to-End Data Flow

1. Data is ingested via **Dataflow Gen2** into OneLake
2. Data is cleaned and transformed into **Lakehouse layers**
3. Structured data is stored in the **Data Warehouse**
4. User submits a natural language query
5. Vanna AI + LLM convert query → SQL
6. RAG layer provides schema context
7. SQL query executes on Fabric warehouse
8. Results are returned and visualized

Outcome

This architecture enables:

Natural language access to complex financial datasets